

Canonical User and Permission Services

Developer guide for new code, migrated code, and legacy façades

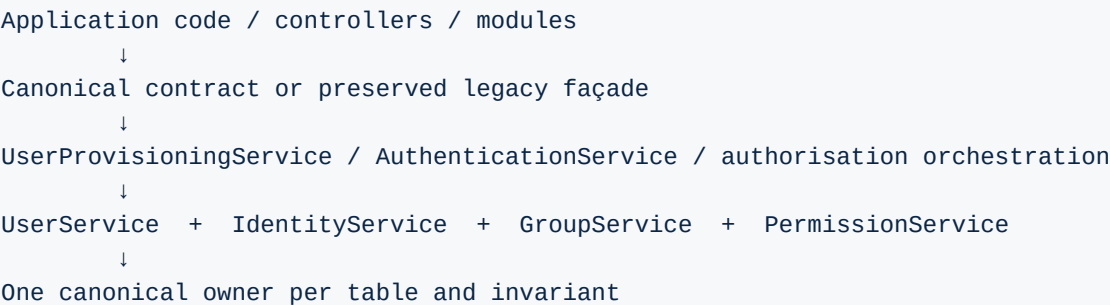
Architectural rule. Services define the contract. Consumers migrate to the contract. Services change only for a proven, generally valid missing capability.

1. Purpose and scope

Chisimba’s canonical security layer separates four kinds of data that legacy code often treated as one object: the application user, the permission identity, group membership, and effective authorisation. The separation gives each table and invariant one owner, while orchestration services coordinate multi-step operations.

This guide explains the intended boundaries, the normal developer call paths, the legacy compatibility façades already used during migration, and the construction-cycle rule discovered during the July 2026 integration checkpoint.

2. The model in one view



Concept	Meaning	Canonical responsibility
User	The Chisimba application account and profile record.	UserService owns user-record reads and writes.
Permission identity	The security identity linked to a user and used by the authorisation model.	IdentityService owns identity creation, lookup and linkage.
Group	Membership such as Guest or Site Admin.	GroupService owns group and membership

Concept	Meaning	Canonical responsibility
membership		behaviour; lower-level leaf stores may support cycle-free internals.
Permission / right	A named capability granted through the authorisation model.	PermissionService and the canonical authorisation path own permission evaluation and assignment.
Provisioning	A transaction spanning user, credential, identity and baseline membership.	UserProvisioningService coordinates the owners; it does not replace them.
Authentication	Credential verification and canonical session creation.	AuthenticationService owns the authentication contract.

3. Ownership and orchestration

A canonical service is not simply a new class name around old SQL. It is the authoritative contract for a domain. Consumers must not reproduce the service's writes, table joins, default memberships or compensation logic.

- Each database table has exactly one canonical owner.
- A service publishes behaviour and return contracts; callers do not depend on its storage details.
- A multi-domain operation is coordinated by an orchestration service.
- Orchestration calls canonical owners; it does not take ownership of their tables.
- A missing consumer convenience is not, by itself, evidence that a service contract should change.

4. Canonical services

4.1 UserService

Use UserService for canonical user lookups and user-record operations. Code should ask the service for a user by a stable identifier or username instead of querying `tbl_users` directly. Preserve the service's published result shape rather than assuming legacy FALSE, empty-string or partially populated-array conventions.

Important integration finding. Delegating the extremely low-level legacy `user::getUserId()` method to UserService caused an object-construction cycle. That migration was rolled back. The capability belongs in UserService, but not every legacy seam is safe to redirect.

4.2 IdentityService

IdentityService owns the permission identity and its linkage to the application user. A consumer that needs an identity must request it through this service. It must not insert or repair identity rows directly.

4.3 GroupService

GroupService owns group definitions and membership behaviour, including baseline groups supplied by policy data. Callers must not hard-code group identifiers, duplicate Guest or Site Admin creation, or write membership tables directly.

4.4 Permission and authorisation services

Permission evaluation belongs to the canonical authorisation path. Controllers and modules ask whether the current identity has a capability; they do not infer rights from usernames, UI visibility, or ad hoc table queries. Mutating operations must still be POST-only, CSRF-protected, permission-checked and validated.

4.5 UserProvisioningService

UserProvisioningService is the canonical transaction for local account creation. The verified `createLocalUser()` chain coordinates the modern credential, user record, permission identity and Guest membership, with compensation when a later step fails. Callers must use this orchestration rather than assembling the steps themselves.

```
createLocalUser(request)
  → validate provisioning input
  → create canonical user and modern credential
  → create/link permission identity
  → assign baseline Guest membership
  → compensate safely if the chain cannot complete
  → return the canonical storage identifier / failure contract
```

5. How developers should use the services

5.1 New code

1. Identify the domain operation: user lookup, identity lookup, group membership, permission evaluation, authentication, or provisioning.
2. Call the canonical service that owns that behaviour. For multi-domain creation, call the orchestration service.

3. Treat the published input, output and failure contract as authoritative.
4. Keep presentation effects—redirects, flash messages, content directories and view rendering—outside the canonical domain service.
5. Add focused tests for the contract and for the consuming controller or module.

5.2 Existing modules

Migrate existing consumers in bounded groups. First identify whether a call is a genuine Chisimba account/security operation or merely a module-owned method with a similar name. Preserve externally relied-upon signatures while replacing the internal implementation, then test before moving to the next shared operation.

- Prefer one high-leverage shared façade migration over edits to many callers.
- Include active, `__PAUSED` and `__PARKED` callers in the evidence inventory; exclude `__OBSOLETE` from migration.
- Label inactive categories separately so they do not distort active-code coverage.
- Do not introduce legacy bridges merely to make an unmigrated consumer appear complete.

5.3 External identity connectors

LDAP, OAuth and similar connectors belong in plugins, not in the canonical core. A connector plugin may translate an external identity into canonical authentication or provisioning requests, but it must not write Chisimba user, credential, identity, group or permission tables directly.

```
External connector plugin
→ canonical authentication / provisioning contract
→ UserService + IdentityService + GroupService + permission services
```

6. Legacy façades and compatibility boundaries

Legacy façades are migration seams, not alternate owners. They preserve a widely used public method long enough for callers to inherit canonical behaviour. Their internals delegate to the canonical contract, and their remaining presentation or file-system side effects stay explicit.

Legacy seam	Current treatment	Developer guidance
<code>useradmin_model2::addUser(...)</code>	Preserves the 13-argument signature, storage-ID return and content-folder side effect. User, credential, identity and Guest-membership writes delegate to <code>UserProvisioningService</code> .	Existing callers may continue temporarily. New code should call the canonical provisioning contract directly.
<code>user::getUserId(\$username)</code>	Canonical delegation was attempted, produced a	Do not reapply the redirect.

Legacy seam	Current treatment	Developer guidance
	construction cycle, and was rolled back to the pre-migration implementation.	Migrate higher-level consumers to UserService or first establish a demonstrably cycle-free leaf seam.
Group-related legacy models	Unsafe eager canonical-service construction was removed where it caused cycles; cycle-free leaf dependencies may be used internally.	Do not instantiate GroupService inside a commonly constructed legacy model without proving the dependency graph.
Legacy connector flags / LDAP sentinel accounts	Rejected rather than preserved in canonical local provisioning.	Implement future connectors as plugins using canonical contracts.

7. Dependency direction and permanent cycle prevention

Permanent construction rule. Legacy façade and model constructors may load only lightweight database or leaf dependencies. They must never eagerly construct canonical orchestration services.

The normal Chisimba object factory constructs many legacy objects indirectly. An eager service lookup in `init()` can therefore form a circular graph even when the requested business operation is never called. PHP eventually reports memory exhaustion in `Engine` or `dbtable`; that line is the exhaustion point, not the root cause.

Unsafe:

legacy model `init()` → orchestration service → legacy dependency → same model

Safe for infrequent commands:

legacy method → method-local orchestration-service resolution → canonical operation

Safe for hot reads:

legacy method → proven cycle-free leaf dependency

- Resolve `UserProvisioningService` inside `addUser()`, not in `useradmin_model2::init()`.
- Do not solve recursion by increasing PHP `memory_limit`.
- Do not broadly make every lookup lazy; first establish coherent dependency direction.
- Add a construction smoke test for common user, user-admin, group, toolbar and permission objects.
- Make the test fail on recursion, excessive construction depth or memory exhaustion.

8. Security and transaction rules

- Use modern password hashes only for a new installation; do not preserve legacy hashes as an architectural requirement.
- Authentication must create the canonical session deliberately and logout must use native POST plus CSRF protection.
- Every mutation must be POST-only, CSRF-protected, permission-checked and safely validated.
- Do not reveal account existence or credential details through inconsistent failures.
- Canonical provisioning must compensate or roll back partial work; callers must not repeat the transaction.
- All new user-visible text belongs in the owning module's language entries.

9. Migration workflow for maintainers

6. Capture current source and exact callers from the repository working tree.
7. Separate genuine domain calls from misleading same-name module methods.
8. Choose one bounded façade or consumer family.
9. Define the preserved legacy signature and the canonical contract it maps to.
10. Check constructor and `init()` dependency direction before applying the delegation.
11. Use method-scoped guards where the file already contains unrelated work.
12. Lint and run focused architectural assertions.
13. Rebuild the disposable runtime from the current working tree; never patch runtime files directly.
14. At integration checkpoints, preserve the database when appropriate and record failures before changing more source.
15. Remove temporary scaffolding and its references when the bounded migration is proven complete.

10. Current verified checkpoint — 30 July 2026

Item	Verified state
Shared user creation	p80 migrated <code>useradmin_model2::addUser()</code> to <code>UserProvisioningService</code> for 18 identified calls across 13 caller files.
Construction-cycle repair	p87 retained canonical <code>addUser()</code> while moving <code>UserProvisioningService</code> resolution from <code>init()</code> into <code>addUser()</code> .
Shared username lookup	p85 delegation of <code>user::getUserId()</code> was removed by p86 because it caused a construction cycle.
Runtime	Rebuilt from the repaired complete working tree with the database and installed configuration preserved.

Item	Verified state
Application	Site reported working again after p87.
Scope boundary	No further consumer migration should occur before the constructor-cycle audit and regression test.

11. Definition of done for a migrated legacy seam

- The canonical service owns every domain read or write covered by the seam.
- The legacy signature and documented return contract are preserved only where current callers require them.
- No direct duplicate writes, hidden bootstrap, connector sentinel or temporary compatibility path remains.
- Construction is cycle-free under the normal 256 MB memory limit.
- Focused tests and a preserved-data integration checkpoint pass.
- Obsolete compatibility code is removed unless it has ongoing architectural value.

Appendix A — next-session continuation brief

Continue the Chisimba Revival from the verified 30 July 2026 checkpoint. The site is working after p87. Preserve p80's canonical `useradmin_model2::addUser()` delegation, including its method-local lazy resolution of `UserProvisioningService`. Preserve p86's rollback of the cycle-inducing `user::getUserId()` delegation. Preserve the current database and installed configuration.

The next bounded task is not another consumer migration. Audit only constructors and `init()` methods touched during the canonical user, identity, group and permission service migration for eager construction of orchestration services. Establish the dependency graph, correct only proven unsafe eager lookups, and add a focused construction smoke test that instantiates the common user, `useradmin_model2`, group, toolbar and permission objects under the normal 256 MB memory limit.

After the focused source change and test, rebuild the disposable PHP 8.2 runtime from the current repository working tree with `--preserve-db`. Verify derek login, Site Admin rights, UserAdmin, editing an existing user, and creation of one temporary user through the lazy canonical provisioning path. Record failures before making any further migration. Do not raise `memory_limit`, patch the runtime, broaden the audit, restore whole files, or disturb unrelated uncommitted work.